

CSCE 636 DEEP LEARNING · MODULE 01 · LECTURE NOTES

Why Depth

Learning the representation instead of designing it.

MODULE OUTCOMES

- Explain why universal approximation does not justify depth.
- Explain architecture as encoded assumption.
- Explain what actually changed around 2012.
- State when deep learning is the wrong choice.
- Set up the tooling and the baseline.

1 The proposal

THE PIPELINE COLLAPSED INTO ONE OPTIMISATION

The classical pipeline had two stages: hand-designed features, then a simple model. SIFT or HOG descriptors feeding a support vector machine; MFCCs feeding a Gaussian mixture; n-gram counts feeding logistic regression. **The feature stage was where the domain expertise lived**, and it was the product of years of careful work.

Each stage was designed and tuned separately, by different people, against proxy objectives that were not the final task — a descriptor was evaluated on whether it was distinctive and repeatable, not on whether it helped the classifier.

Deep learning replaces the entire pipeline with a single function trained end to end on the actual objective. The early layers learn whatever representation serves the final loss, and there is no intermediate target to approximate.

So the features are optimised for the task rather than for a human's theory of what should matter — and that turns out to be worth more than decades of hand design, **wherever there is enough data to learn the representation from.** That qualifier carries the whole argument, and §4 is about where it fails.

UNIVERSAL APPROXIMATION DOES NOT EXPLAIN ANY OF THIS

The theorem: a feedforward network with a single hidden layer and a non-polynomial activation can approximate any continuous function on a compact set to arbitrary accuracy. It is true and it is cited constantly as a justification for neural networks.

It says nothing about how many hidden units are required — which can be exponential in the input dimension — **nothing about whether gradient descent will find those weights**, and **nothing about whether the result generalises to new data**. All three are the questions that matter.

And it does not justify depth at all. It is a statement about shallow networks: one hidden layer already suffices. If the theorem were the reason deep learning works, deep learning would not need to be deep.

What depth actually buys is efficiency of representation. There are functions a deep network expresses with a modest number of units that require exponentially many units to express with one hidden layer — compositional and hierarchical functions especially. **Expressivity was never the binding constraint; learnability was**, and that is an optimisation question (Module 03) rather than an approximation-theoretic one.

2 Architecture as encoded assumption

Architecture	What it assumes about the data	So it suits
Fully connected	Nothing at all. Every input may relate to every other, with no structure assumed.	Tabular data — where it loses to gradient-boosted trees (CSCE 633 Module 06 §3), because assuming nothing means learning everything from examples.
Convolution	Locality and translation invariance — nearby inputs relate, and a pattern means the same thing wherever it appears.	Images, audio spectrograms, and anything on a regular grid where both assumptions genuinely hold.
Recurrence	Sequential dependence, with the same dynamics applied at every step.	Sequences — and largely superseded by attention (Module 06).
Attention	Any element may relate to any other, and which ones do is learned rather than fixed by the architecture.	Text, and increasingly images, audio, and proteins — a weaker prior that needs more data and scales further (Module 07).
Graph networks	The relations are given explicitly by a graph the data comes with.	Molecules, meshes (CSCE 645), social networks, scene graphs.
Equivariant networks	A known symmetry group — rotation, permutation, translation — under which outputs should transform predictably.	Physics, chemistry, 3D geometry, where the symmetry is a known fact rather than a guess.

A MATCHED INDUCTIVE BIAS IS WORTH ENORMOUS AMOUNTS OF DATA

A fully connected layer applied to a 224×224 colour image has 150,000 inputs and no notion whatsoever that adjacent pixels are related. It must discover locality from examples — and it can, given enough of them, which is the expensive part.

A convolution builds that knowledge in. Weights are shared across spatial positions, so a feature learned at one location is available at every location, and the parameter count drops by orders of magnitude for the same expressive power on the functions that matter.

That is not merely a computational efficiency — it is a *correct assumption* about images, and a correct assumption substitutes directly for data. The model does not need to spend examples learning something it was told.

Which is exactly why the same architecture loses on tabular data. Column order in a spreadsheet is arbitrary, so locality is false, and a convolution over columns encodes an assumption that is not merely absent but *wrong*. **Architecture is a prior, and a wrong prior is worse than a weak one** — which is the cleanest way to understand CSCE 633 Module 06's result about trees.

3 What actually changed

Ingredient	When	Why it mattered
Backpropagation	1986 (popularised)	The mathematics was never the bottleneck. It was available for twenty-five years before the results arrived.
Convolutional networks	1989	LeCun's digit recognition worked and was deployed on cheques — at a scale that did not generalise to harder problems.
GPUs	roughly 2007–2012	Two orders of magnitude of arithmetic throughput , and the workload is almost entirely matrix multiplication (CSCE 735 Module 08). The single largest factor.
Large labelled datasets	ImageNet, 2009	Fourteen million labelled images — enough to learn a representation rather than merely fit a classifier.
ReLU	~2011	Fixed vanishing gradients in deep stacks (Module 02 §4). A one-line change with disproportionate effect.
Principled initialisation	2010–2015	Xavier and He initialisation. Made networks beyond a few layers trainable at all , which they previously were not.
Normalisation layers	2015	Batch normalisation made training robust to the learning rate and much faster to converge (Module 04).

The idea waited twenty-five years for the compute and the data — and also for a handful of small engineering fixes that mattered enormously. **It is a useful corrective to the 'it was just compute' story** that ReLU, initialisation, and normalisation were each decisive, each simple, and each took years to find. **Most of this course is about that kind of fix**, which is why Modules 03 and 04 are longer than Module 02.

4 When deep learning is the wrong choice

- **Tabular data.** Gradient-boosted trees usually win (CSCE 633 Module 06 §3), with far less tuning, no GPU, and native handling of missing values and categoricals. **§2 explains why:** there is no structure for an architecture to exploit.
- **Small data.** Below a few thousand examples with no suitable pretrained model available, **there is nothing to learn a representation from** — and the whole proposal of §1 depends on there being.
- **You need to explain the decision.** **Attribution methods are not explanations** (Module 12 §4), and in regulated settings a linear model with readable coefficients may be required rather than merely preferable.
- **Inference cost matters more than accuracy** — on a microcontroller, inside a 16 ms frame budget (CSCE 650), or at a scale where energy is the binding constraint. Module 13 covers what can be recovered by quantisation and distillation, and it is not unlimited.

- **Or a classical method already solves the problem — the most common case and the least often checked. Fit the CSCE 633 baseline first and keep it;** it is the number every result in this course is reported relative to, and occasionally it wins.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Karpathy — Neural Networks: Zero to Hero, lecture 1 (free)	Start here this week. Builds autodifferentiation from nothing on video; Module 02 assumes you have watched it.
Goodfellow, Bengio & Courville — Deep Learning, chapters 1 and 6 (free)	The §1 framing and the universal approximation discussion, stated carefully rather than as a slogan.
Krizhevsky, Sutskever & Hinton — ImageNet Classification with Deep CNNs (free)	The 2012 result of §3. Read it for the engineering detail — the architectural choices are all justified by ablations.
Battaglia et al. — Relational inductive biases, deep learning, and graph networks (free)	The §2 framing, stated as a general principle. The clearest available treatment of architecture as prior.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Fit the CSCE 633 baseline** on the dataset you will use all semester and record it.
- 2 Train a fully connected network on tabular data and compare against gradient boosting. **Report which wins.**
- 3 **Train the same network on images with the pixels randomly permuted** (a fixed permutation). Compare against a CNN on the unpermuted images, and against a CNN on the permuted ones.
- 4 Explain the three results using §2's argument.
- 5 **Count the parameters** of a fully connected layer and a convolutional layer with the same input and output sizes.
- 6 Train a network with sigmoid activations and with ReLU at eight layers deep. Report what happens.
- 7 **Train with and without a principled initialisation** at ten layers.
- 8 Measure the wall-clock difference between CPU and GPU training for one epoch.
- 9 Find a problem where a classical method beats a network, and document it.

- 10 Set up experiment tracking and a tiny overfittable dataset before Module 02.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does deep learning propose, and what does it replace?
- 2 State the universal approximation theorem and three things it does not say.
- 3 What does depth actually buy?
- 4 Give six architectures and the assumption each encodes.
- 5 Why does a correct inductive bias substitute for data?
- 6 Why does convolution lose on tabular data?
- 7 Name seven ingredients and say which was the largest factor.
- 8 Give five cases where deep learning is the wrong choice.